

Data Structures and Algorithms

Lecture 2: Linked Lists

Data Science

Beijing Normal-Hong Kong Baptist University

Outline

- Abstract Data Type (ADT)
- List ADT
- Linked lists

Abstract Data Type (ADT)

- Data type = Data + Operation
 - Example 1: **integer**
 - Data: a whole number
 - Operations: +, -, *, /, ...
 - Example 2: **string**
 - Data: an array of characters
 - Operations: strlen, strcpy, strcat, strcmp, ...
- Can this be generalized?
 - Abstract Data Type (ADT)
 - Encapsulation

Encapsulation - What



- Users of data
 - do not touch data directly
 - operates on data by calling the methods
 - do not know how the methods are implemented

Encapsulation - Why

- **Modular**: one module for one ADT
 - Implementation of the ADT is **separate** from its use
 - Allows parallel development
 - Easier to debug
- Code for the ADT can be **reused** in different applications
- **Information hiding**
 - Protect data from unwanted operations
 - implementation details can be changed without affecting user programs
- Allow rapid **prototyping**
 - Prototype with simple ADT implementations, then tune them later when necessary

Encapsulation - How

- In OOP Languages:
 - ADT: Class
 - Data: member variables
 - Operations: member functions
- In C:
 - Data: variables (usually of a struct data type)
 - Operations: functions
 - *Information hiding is not explicitly supported in C*

The List ADT - Data

- A **sequence** of zero or more elements

$$A_1, A_2, \dots, A_N$$

- N : length of the list
- A_1 : first element
- A_N : last element
- A_i : element at position i
- If $N = 0$, then it is an empty list
- Linearly ordered
 - A_i precedes A_{i+1}
 - A_i follows A_{i-1}
- The elements can be of any data type but we use **double** for discussion

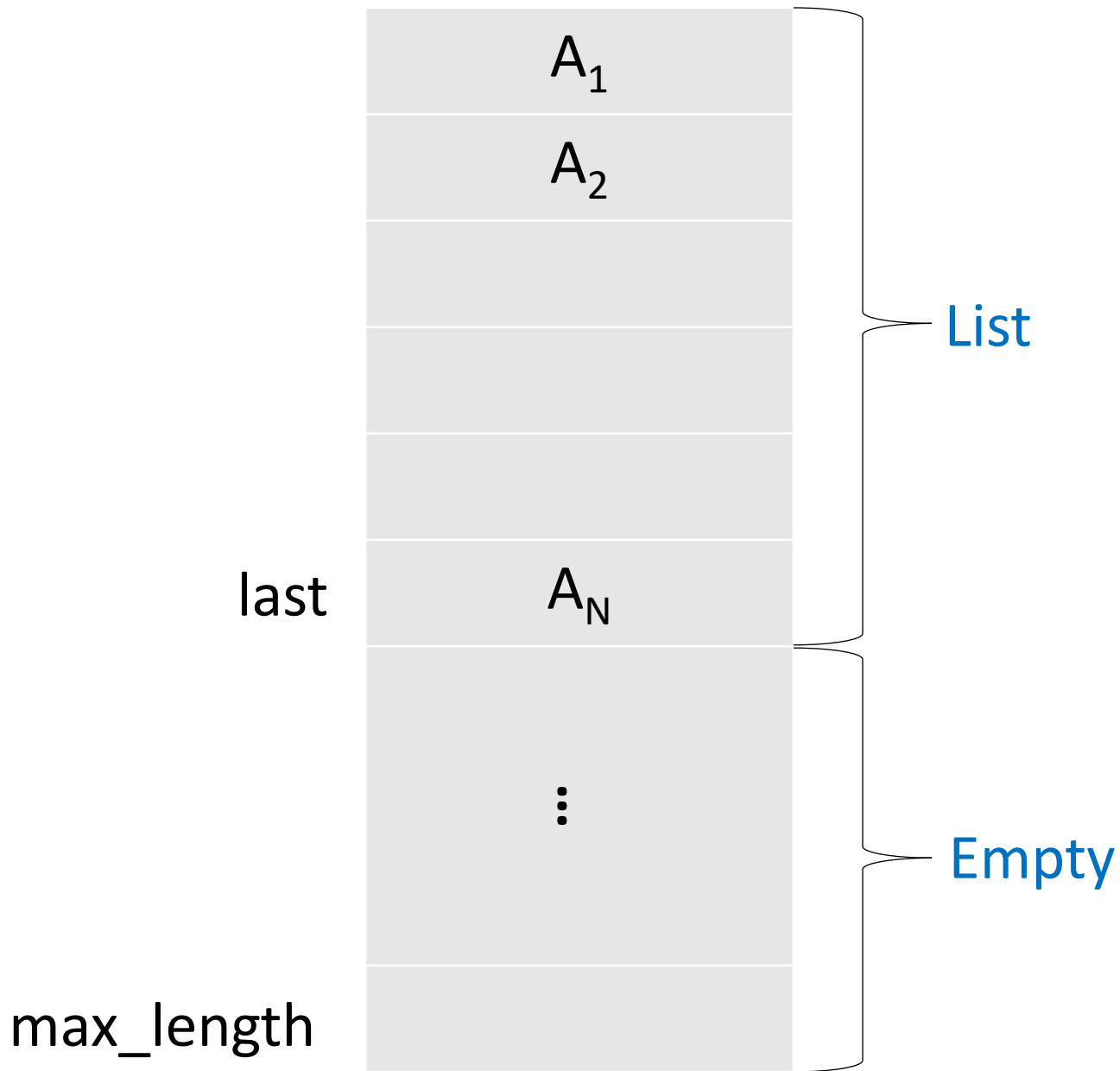
The List ADT - Operations

- **makeEmpty**: create an empty list
- **insert**: insert an object to a list (insert after the given position, starts at 1)
 - List: 34, 12, 52, 16, 12
 - `insert(x, 2) -> 34, 12, x, 52, 16, 12`
- **remove**: delete an element from the list
 - List: 34, 12, 52, x, 16, 12
 - `remove(52) -> 34, 12, x, 16, 12`
- **find**: locate the position of the first occurrence of an element in the list
 - List: 34, 12, x, 16, 12
 - `find(12) -> 2`
- **findKth**: retrieve the element at a certain position
 - List: 34, 12, x, 16, 12
 - `findKth(3) -> x`
- **printList**: print the list

Implementation of an ADT

- Define data using **data types**
- Define operation using **functions**
- Two standard implementations for the list ADT
 - Array-based
 - Linked list

Array Implementation

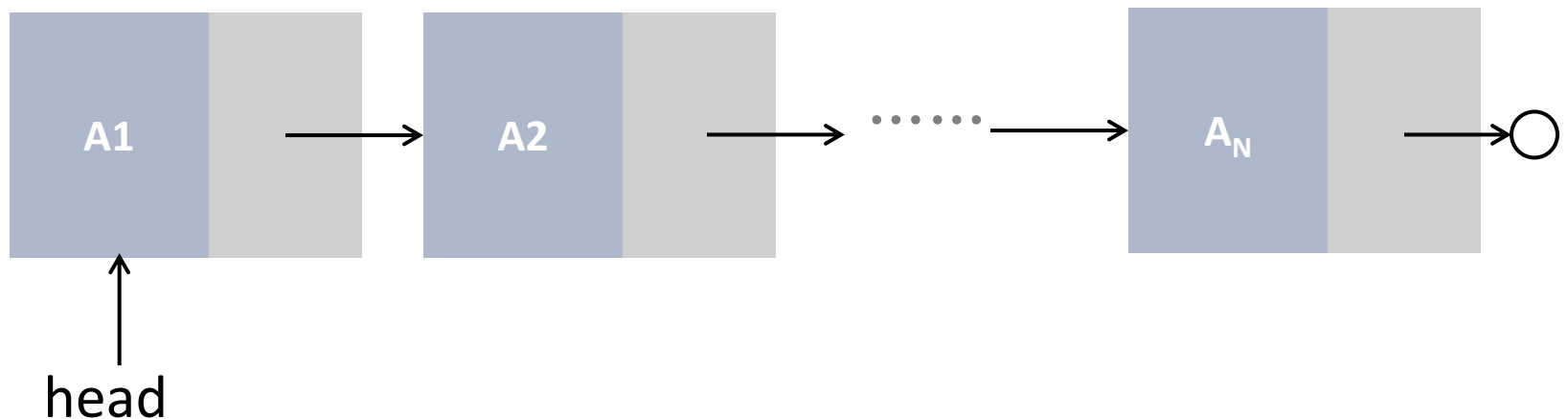


Discussion

- Are these operations **suitable** for the array implementation?
 - `printList`
 - `find`
 - `insert`
 - `remove`
 - `findKth`
- Any additional **pros** and **cons**?

Pointer Implementation (Linked List)

- Ensure that the list is not stored contiguously
 - A node stores one element
 - The address of a node is stored in its **previous node**
 - The address of the **first node** must be stored



Pointer Implementation-Example



Pointer Implementation-Example



Second-phase campus (under construction)
二期校园 (建设中)

Sports Park
体育公园

Campus Guide
校園

Bob
T4-501

T8-302

Arts Hill
艺薈

Excellence Road
至道路

Rural Creativity Clusters
文化创意群落

Learning Resource Centre
学习资源中心

Performance Theatre
演艺厅

Administration Building
行政楼

University Hall
大学会堂

Sports Complex
体育馆

Jintong Road
金同路

The Great Learning Way
大学道

Engagement Road
承民路

Huitong Village
会同村

Teaching Area
教学区

Bo Ya Town (Residential Area)
博雅小镇 (宿舍区)

T1 Business and Management Building
厚德楼

T2 Humanities Building
人文楼

T3 Science Building
格物楼

T4 Teaching Building
教学楼

T5 Teaching Building
教学楼

T6 Teaching Building
教学楼

T7 Teaching Building
教学楼

T8 CEFC Building
博英楼

T29 Teaching Building
教学楼

V15 Ming Xin House
明心楼

V16 Shang Xian House
尚贤楼

V17 Ren Ai House
仁爱楼

V18 Bao Pu House
抱朴楼

V19 Zhen Zhi House
真知楼

V20 Du Xing House
笃行楼

V21 Bo Wen House
博文楼

V22 Ya Zhi House
雅志楼

V23 Xue Qin House
雪芹楼

V24 Yang Ming House
阳明楼

V25 Dong Po House
东坡楼

V26 Goethe House
歌德楼

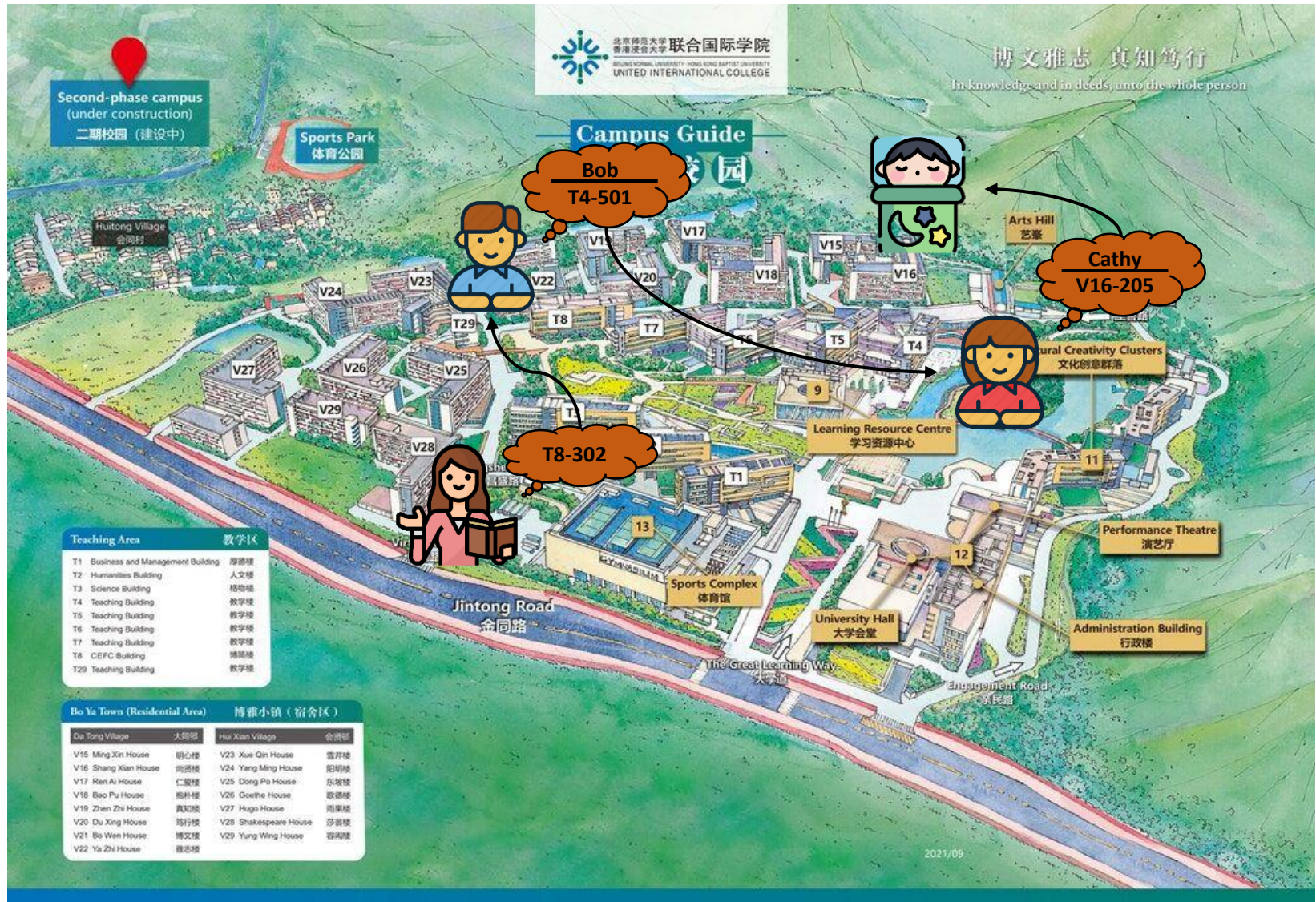
V27 Hugo House
雨果楼

V28 Shakespeare House
莎翁楼

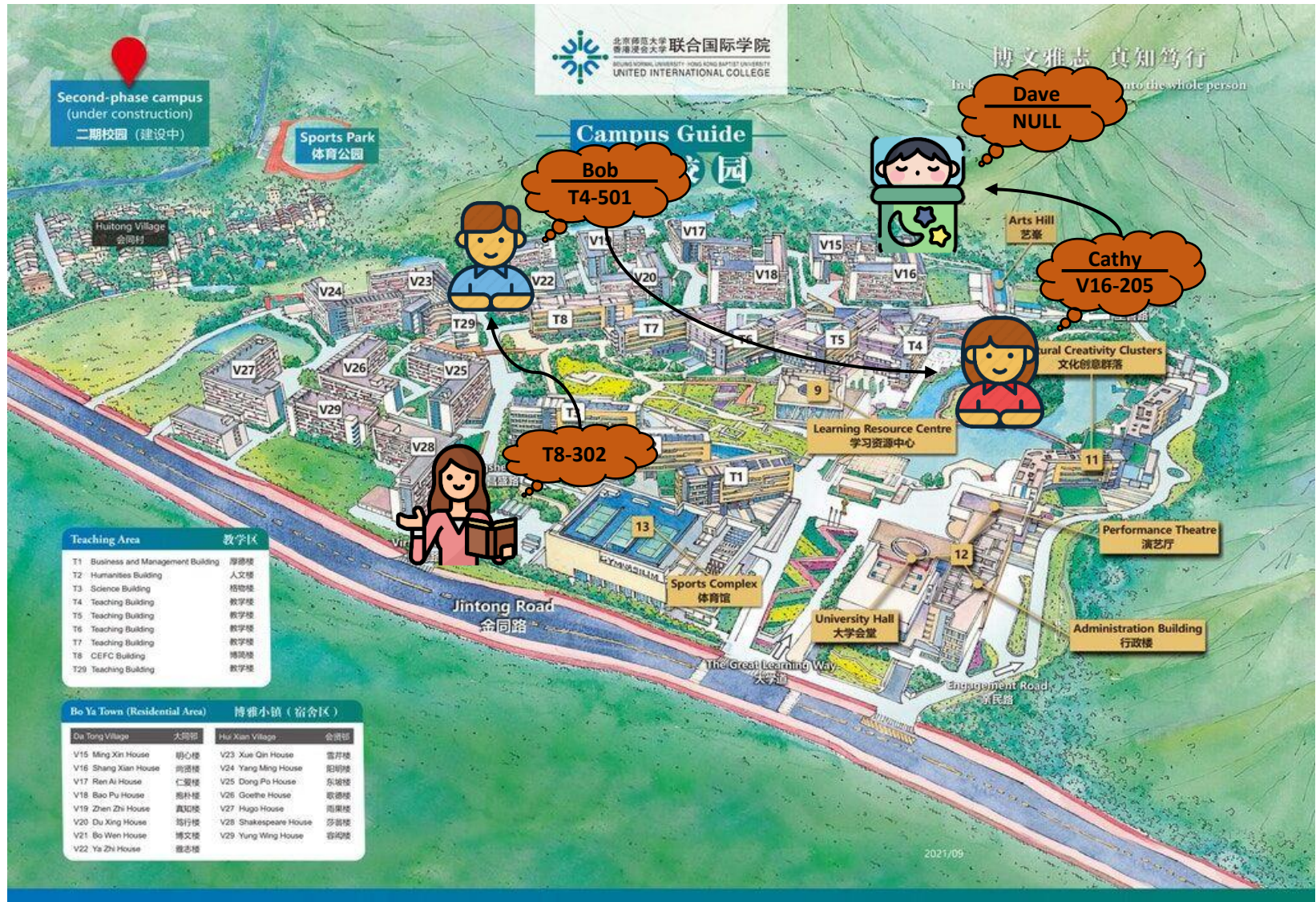
V29 Yung Wing House
容闳楼

2021/09

Pointer Implementation-Example



Pointer Implementation-Example



Discussion

- Are these operations **suitable** for the linked list?
 - printList
 - find
 - findKth
 - insert
 - remove
- Any additional **pros** and **cons**?

A Complete list of Comparison

Topic		Array	Linked List
Efficiency	insert		
	remove		
	find		
	findKth		
space			

Linked List Implementation

- Data
 - Nodes

```
typedef struct node{  
    double data;  
    struct node* next;  
} Node;
```

- A pointer to the first node

```
Node* head;
```

Linked List Implementation

- Methods

```
bool IsEmpty(Node* head);  
Node* InsertNode(Node** phead, int index, double x);  
int FindNode(Node* head, double x);  
int DeleteNode(Node** phead, double x);  
void DisplayList(Node* head);  
void DestroyList(Node** head);
```

Methods

- `bool IsEmpty(Node* head)`
 - returns `true` if the list is empty and *false* otherwise
- `Node* InsertNode(Node ** phead, int index, double x)`
 - insert a new node after position `index`
 - position of nodes starts from 1
 - insert a new node as the head if `index=0`
 - returns a pointer to the `new node` if insertion is successful and `NULL` otherwise
- `int FindNode(Node* head, double x)`
 - returns the position of the first node whose `data=x`
 - returns 0 if no such node exists

Methods (Cont.)

- `int DeleteNode(Node** phead, double x)`
 - deletes the first node whose `data=x`
 - returns the position of the deleted node
 - returns 0 if no such node exists
- `void DisplayList(Node* head)`
 - prints all the nodes in the list
- `void DestroyList(Node** head)`
 - deletes all the nodes in the list
 - frees the memory allocated to the nodes
 - Point the original head to NULL

Insert

- `Node* InsertNode(Node** phead, int index, double x)`
 - insert a new node after position `index`
 - position of nodes starts from 1
 - insert a new node as the head if `index=0`
 - returns a pointer to the new node if insertion is successful and *null* otherwise
- Why `Node **phead`?

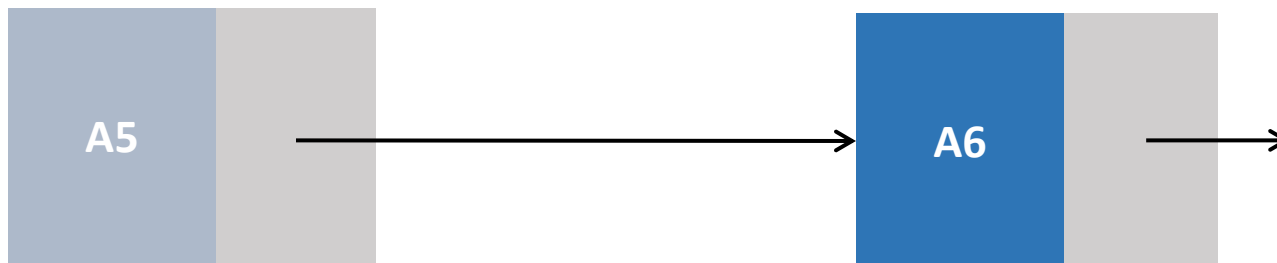
Insert

- Locate the element at position index
- Allocate memory for the new node
- Point the new node to its successor
- Point the new node's predecessor to the new node

Insert

- **Locate** the element at position index
- **Allocate memory** for the new node
- Point the new node to its **successor**
- Point the new node's **predecessor** to the new node

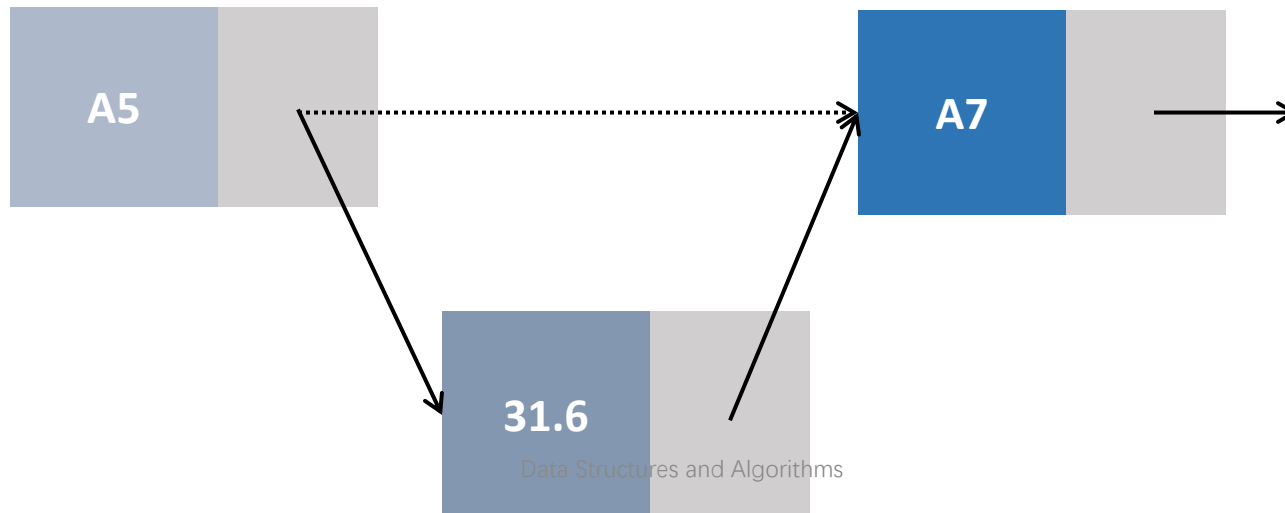
InsertNode(&head, 5, 31.6)



Insert

- **Locate** the element at position index
- **Allocate memory** for the new node
- Point the new node to its **successor**
- Point the new node's **predecessor** to the new node

InsertNode(&head, 5, 31.6)



Insert

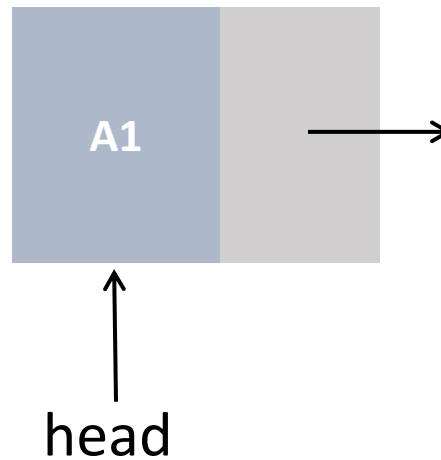
- Possible cases of `InsertNode`
 - Insert into an empty list
 - Insert in front
 - Insert at back
 - Insert in middle
- But, in fact, only need to handle two cases
 - Insert as the first node (Case 1 and Case 2)
 - Insert in the middle or at the end of the list (Case 3 and Case 4)

Two Cases for Insert

- Insert as the first node
 - handles the **next** pointer of **one node**
 - updates the **head** pointer
- Insert in the middle or at the end
 - handles the **next** pointer of **two nodes**

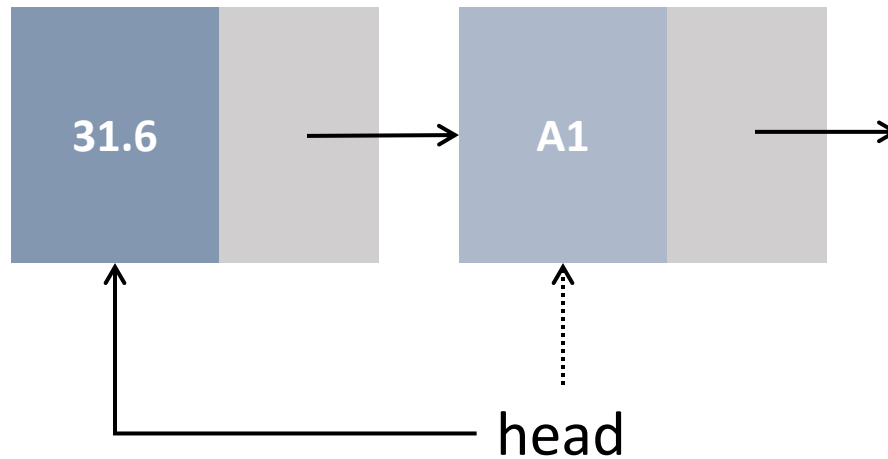
Insert as the First Node

`InsertNode(&head, 0, 31.6)`



Insert as the First Node

`InsertNode(&head, 0, 31.6)`



Code for Insert

```

#include <stdlib.h>
Node* InsertNode(Node** phead, int index, double x) {
    if (index < 0) return 0;

    int currIndex = 1;
    Node* currNode = *phead;
    while (currNode && index > currIndex) {
        currNode = currNode->next;
        currIndex ++;
    }
    if (index > 0 && currNode == 0) return 0;

    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = x;
    if (index == 0) {
        newNode->next = *phead;
        *phead = newNode;
    }
    else {
        newNode->next = currNode->next;
        currNode->next = newNode;
    }
    return newNode;
}

```

```
#include <stdlib.h>
Node* InsertNode(Node** phead, int index, double x) {
    if (index < 0) return 0;
```

```
    int currIndex = 1;
    Node* currNode = *phead;
    while (currNode && index > currIndex) {
        currNode = currNode->next;
        currIndex ++;
    }
    if (index > 0 && currNode == 0) return 0;
```

Try to locate the node at position *index*. If it does not exist, return *null*.

```
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = x;
    if (index == 0) {
        newNode->next = *phead;
        *phead = newNode;
    }
    else {
        newNode->next = currNode->next;
        currNode->next = newNode;
    }
    return newNode;
}
```

```

#include <stdlib.h>
Node* InsertNode(Node** phead, int index, double x) {
    if (index < 0) return 0;

    int currIndex = 1;
    Node* currNode = *phead;
    while (currNode && index > currIndex) {
        currNode = currNode->next;
        currIndex ++;
    }
    if (index > 0 && currNode == 0) return 0;

    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = x;
    if (index == 0) {
        newNode->next = *phead;
        *phead = newNode;
    }
    else {
        newNode->next = currNode->next;
        currNode->next = newNode;
    }
    return newNode;
}

```

Create a new Node.

```

#include <stdlib.h>
Node* InsertNode(Node** phead, int index, double x) {
    if (index < 0) return 0;

    int currIndex = 1;
    Node* currNode = *phead;
    while (currNode && index > currIndex) {
        currNode = currNode->next;
        currIndex ++;
    }
    if (index > 0 && currNode == 0) return 0;

    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = x;
    if (index == 0) {
        newNode->next = *phead;
        *phead = newNode;
    }
    else {
        newNode->next = currNode->next;
        currNode->next = newNode;
    }
    return newNode;
}

```

Insert as the
new head.

```

#include <stdlib.h>
Node* InsertNode(Node** phead, int index, double x) {
    if (index < 0) return 0;

    int currIndex = 1;
    Node* currNode = *phead;
    while (currNode && index > currIndex) {
        currNode = currNode->next;
        currIndex ++;
    }
    if (index > 0 && currNode == 0) return 0;

    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = x;
    if (index == 0) {
        newNode->next = *phead;
        *phead = newNode;
    }
    else {
        newNode->next = currNode->next;
        currNode->next = newNode;
    }
    return newNode;
}

```

Insert after
currNode.

Find

- `int FindNode(Node* head, double x)`
 - returns the position of the first node whose `data=x`
 - returns 0 if no such node exists
- Steps
 1. Search for a node with the value equal to `x` in the list.
 2. If such a node is found, return its position. Otherwise, return 0.

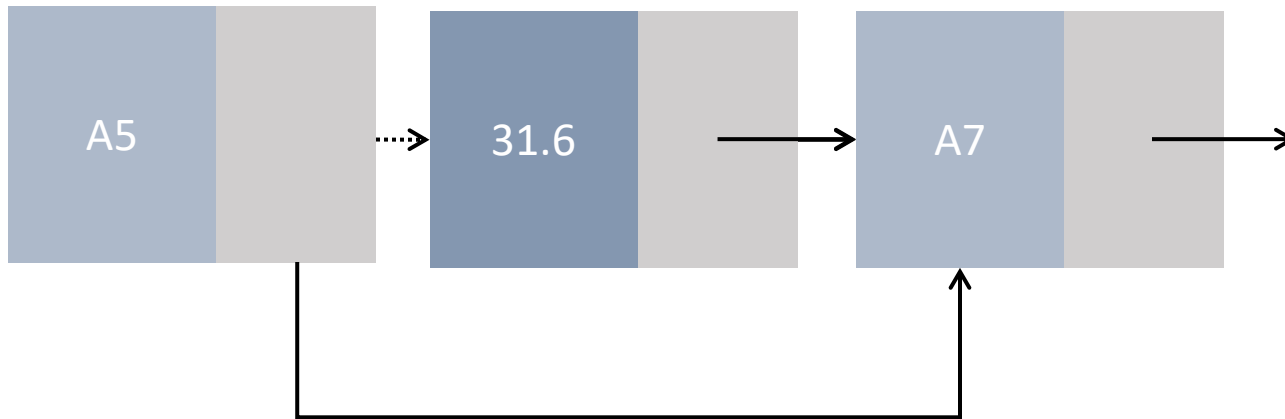
Delete

- `int DeleteNode(Node** phead, double x)`
 - deletes a node whose **data=x**
 - returns the position of the deleted node
 - returns 0 if no such node exists
- Steps
 1. Find the desirable node (similar to **FindNode**)
 2. In addition, record the node's **predecessor**
 3. Set the pointers
 4. **Free the memory** occupied by the found node

Delete

- Deleting a middle or an end node

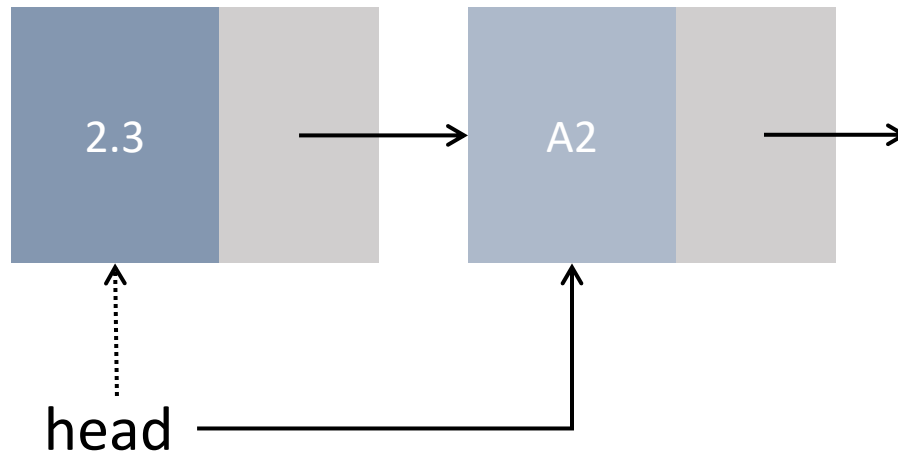
DeleteNode(&head, 31.6)



Delete

- Deleting the head

DeleteNode(&head, 2.3)



Destory

- `void DestroyList(Node** head)`
 - deletes all the nodes in the list
 - frees the memory allocated to the nodes
 - Point the original head to NULL
- Steps
 - Step through the list and delete each node one by one.
 - Before deleting a node, obtain its next node.